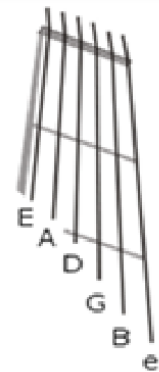


Présentation du projet

A - Rappel sur l'accordage d'une guitare

L'accordage standard d'une guitare à six cordes, de la plus grave à la plus aiguë (de 6 à 1), est : Mi La Ré Sol Si Mi (E A D G B E). Les cordes consécutives d'une guitare sont donc accordées à la quarte juste (deux tons et un demi-ton diatonique) à l'exception de l'écart entre la 3e et la 2e corde où c'est une tierce majeure (deux tons).

Corde	Note	Notation anglophone	Fréquence
1 (la plus fine)	Mi	E	329,6 Hz
2	Si	B	246,88 Hz
3	Sol	G	196,0 Hz
4	Ré	D	146,79 Hz
5	La	A	110,0 Hz
6 (la plus grosse)	Mi	E	82,4 Hz



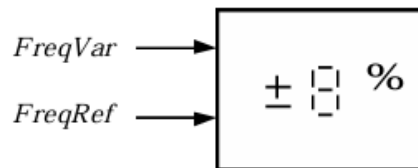
En pratique, pour obtenir une guitare dont les cordes sont accordées les unes par rapport aux autres, il faut que :

- La note donnée par la corde 6 en appuyant sur la 5e case soit égale à la note donnée par la corde 5.
- La note donnée par la corde 5 en appuyant sur la 5e case soit égale à la note donnée par la corde 4.
- La note donnée par la corde 4 en appuyant sur la 5e case soit égale à la note donnée par la corde 3.
- La note donnée par la corde 3 en appuyant sur la 4e case soit égale à la note donnée par la corde 2.
- La note donnée par la corde 2 en appuyant sur la 5e case soit égale à la note donnée par la corde 1.

B – Fonctionnement attendu par le client du projet

Notre client (fictif) souhaite que notre produit ait le fonctionnement décrit ci-dessus [si j'ai changé un élément du design du client, je le mettrai en **gras**] :

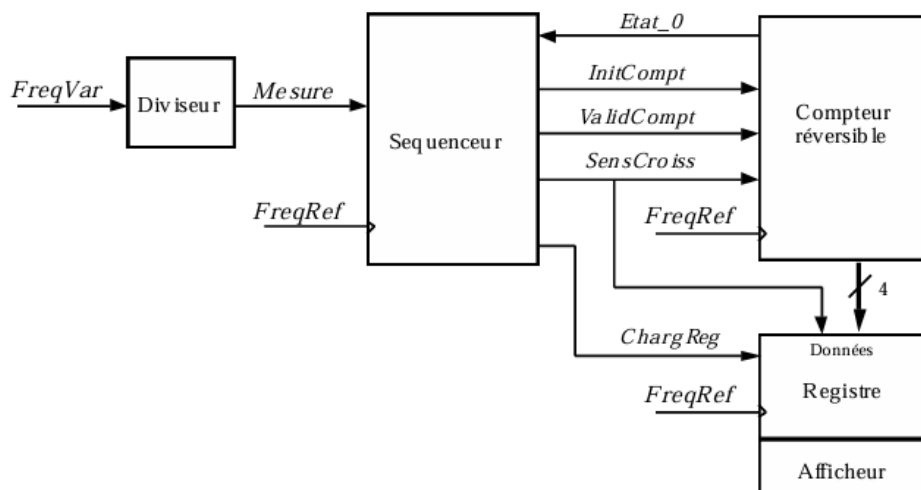
Dans notre cas, l'accord des cordes se fera en effectuant une mesure comparative de la note jouée, avec une fréquence de référence stockée, de façon fixe, en « mémoire ». Pour chacune des notes à accorder, on aura comme objectif, la conception d'un dispositif de mesure et d'affichage de l'écart relatif entre les fréquences de 2 signaux périodiques, comme ce qui suit :



FreqRef est un signal de référence de fréquence fixe,
FreqVar un signal de fréquence variable.

L'écart entre les valeurs des deux fréquences reste faible à tout moment et est supposé ne jamais dépasser 9% (exclu). L'affichage du résultat de la mesure s'effectue en %, sur un chiffre signé.

Un schéma bloc du code FPGA est imaginé :



Le séquenceur, le compteur réversible et le registre disposent d'un même signal d'horloge, Clk.

FreqRef est un signal créé en divisant la fréquence de l'horloge. **Lizandre a décidé de ne pas diviser le signal d'horloge avec plusieurs PLL mais avec des diviseurs par souci de simplicité.** Leur fonctionnement est synchrone.

Un afficheur, de type 7 segments intégré au FPGA, reflète en permanence l'information mémorisée par le registre. Cette information se compose du sens et de la valeur de l'écart (respectivement les valeurs de SensCroiss et des 4 bits de plus faibles rangs du compteur au moment du chargement du registre).

C - Cahier des charges et outils utilisés

Le client veut clairement que l'on utilise une carte FPGA. Nous sélectionnerons la Basys2 et programmerons sur Xilinx 9.2i ISE (non une version plus récente car on ne peut plus modéliser des signaux dans les versions plus récentes). Remarque : on ne peut pas mixer les cartes FPGA de Intel et Altera avec des logiciels de Intel (Quartus) et d'Altera (Xilinx ISE).



On demande que tous les sous-ensembles du FPGA synthétisés doivent être conçus uniquement à partir de :

- Portes NAND à 2, 4 ou 8 entrées
- Bascules D déclenchées sur front montant d'horloge
- Compteurs intégrés (synchrone, 4 bits, code binaire naturel, réversibles, déclenchés sur front montant d'horloge)

En bref, on ne veut aucun latches, multiplexeurs et blocs IP.

PARTIE Numérique

I - Principe de la mesure

Soient T_{ref} et T_{var} les périodes des deux signaux.

Le diviseur délivre un signal Mesure actif pendant une durée de $100 \cdot T_{var}$. Le compteur réversible (de code binaire naturel), initialisé au préalable à l'état d'équivalent décimal 100, est autorisé à évoluer pendant la durée d'activité de Mesure.

Son évolution consiste à décompter jusqu'à ce que l'état 0 soit atteint (ceci nécessite 100 périodes de son horloge, $100 \cdot T_{ref}$), puis à compter depuis cet état.

Si n_f représente l'état atteint par le compteur et $Sens_{Croiss}$ la sélection du sens d'évolution du compteur (0: décomptage, 1: comptage) en fin d'activité de Mesure, le compteur a parcouru $100 + (-1)^{Sens_{Croiss}} \cdot n_f$ états.

$$100 \cdot T_{var} = [100 + (-1)^{Sens_{Croiss}} \cdot n_f] \cdot T_{ref}$$

La valeur de l'écart est donc donnée par $n_f = 100 \cdot \left| \frac{T_{var} - T_{ref}}{T_{ref}} \right|$

Fréquence (Hz)	Pourcentage	Valeur
....		
329,6	+8%	348,890
329,6	0%	379,248
329,6	-8%	409,588
....		
246,8	+8%	465,815
246,8	0%	506,319
246,8	-8%	546,823
....		
196,0	+8%	586,735
196,0	0%	637,755
196,0	-8%	688,775
....		

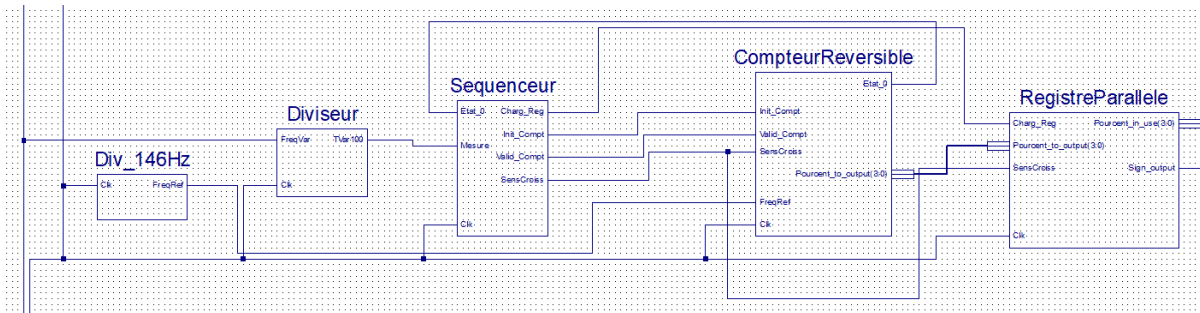
Cette méthode **aurait** pu se décliner plus simplement.

Donc au lieu de créer un compteur commençant à 100 et changeant de sens à 0, on aurait pu juste compter en coup d'horloge et faire la conversion au niveau du registre parallèle (on aura en revanche besoin d'y préciser la note).

Ici est respectivement représenté l'écart relatif, la fréquence associée et le seuil en coup d'horloges (pour une de 50MHz).

II - Fonctionnement interne de chaque bloc

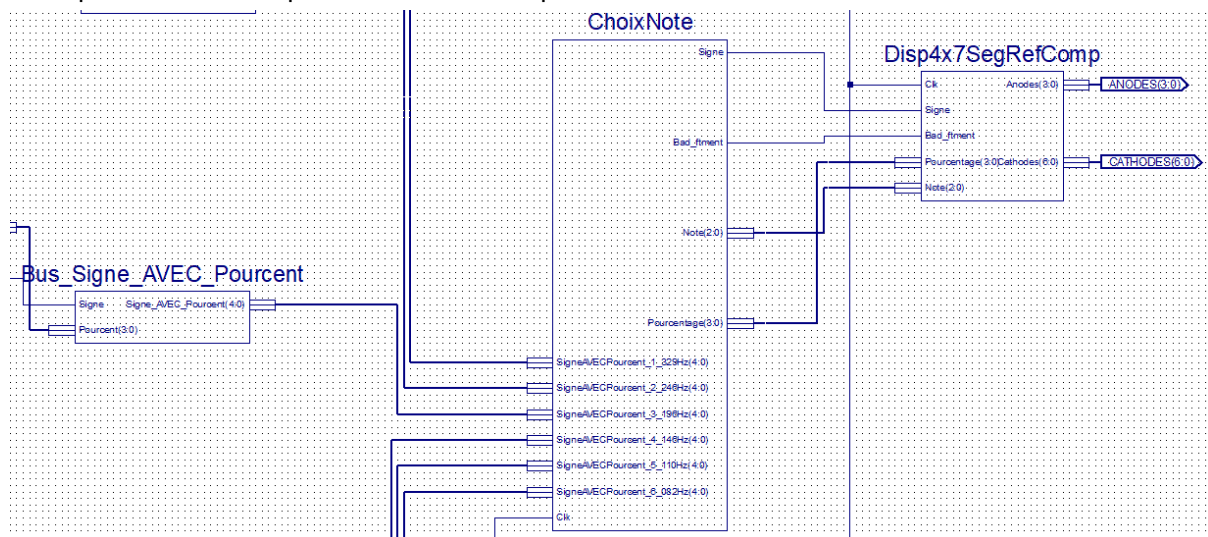
0 – Aperçu global du schéma bloc utilisé en pratique



Dû à la remarque faite plus haut, une seule entrée de type horloge est utilisée. La division de l'horloge se fait via Div146Hz (pour la note à 146Hz) mais est désormais sous piste en cuivre (et non en or). On ne peut plus utiliser rising_edge sur FreqRef.

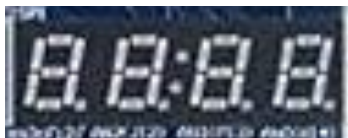
Par soucis de synchronisation et simplification, Lizandre a décidé de mettre l'horloge Clk sur tous les blocs (une solution correcte mais rendant le code moins lisible aurait été de mettre un encodeur à la place de rising_edge(Clk) [S'activer si et seulement si $Q \neq Q_{\text{avant}}$]). Sinon, des latches se créeraient.

Mis à part cela la description du client est respectée.



Le client n'a rien précisé sur la parallélisation et l'affichage. ChoixNote permet de savoir lequel des notes répond et sélectionne son signe et son pourcentage. Vu que les notes sont exclusives, il n'est pas censé y avoir de conflits.

L'affichage utilisé est celle intégré à la Basys2 [On a assez de place pour y placer la note, le signe et le pourcentage]. La traduction des éléments à afficher en cathodes/anodes de l'afficheur se fera dans le bloc Disp4x7SegRefComp.



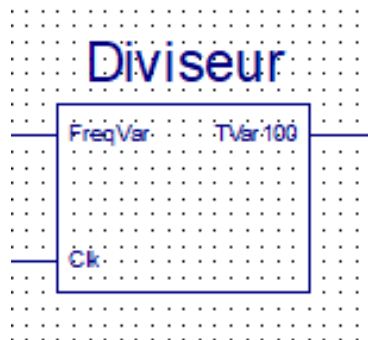
A - Diviseur

1) Fonctionnement attendu par le client

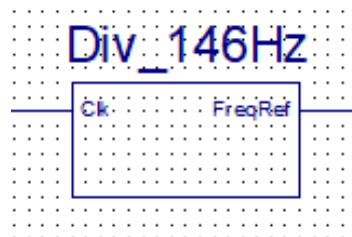


Il génère de façon périodique, permanente, le signal Mesure à partir du signal d'horloge formé par FreqVar. Mesure est actif pendant 100 périodes de cette horloge, et inactif pendant un nombre quelconque (à fixer librement) de périodes.

2) Codage effectué



Ici pour suivre strictement le fonctionnement attendu par le client sans utiliser une horloge, on le code comme si on fabriquait un encodeur.



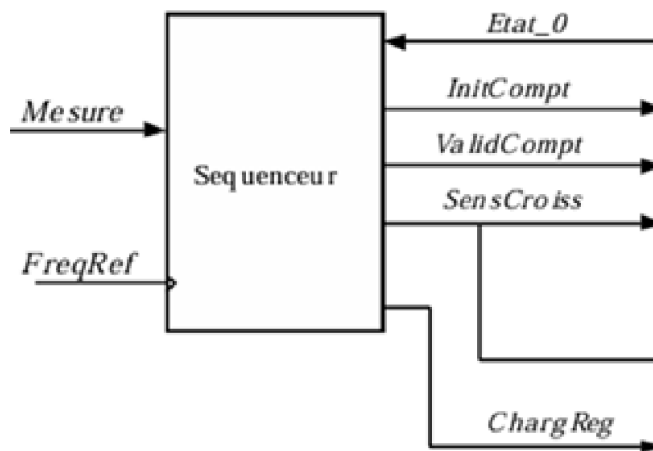
Le but de Div_146Hz est de diviser la fréquence de l'horloge pour fabriquer un signal **enable** de fréquence souhaitée. Ici SEUIL_196Hz = 6378 cela représente 1% de la fréquence relative de 196Hz.

```

41 process(Clk)
42 begin
43     if(rising_edge(Clk)) then
44         if(compteur_reel=SEUIL_196Hz) then compteur_reel<=0; FreqRef <= '1';
45         else compteur_reel<=compteur_reel+1; FreqRef <= '0'; end if;
46     end if;
47 end process;
48 end Behavioral;
  
```

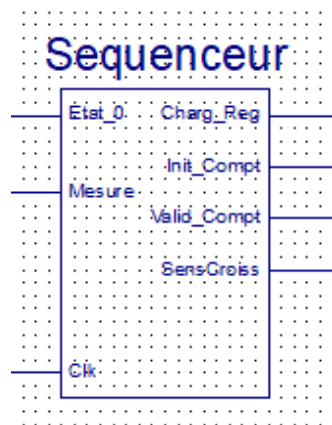

B-Séquenceur

1) Fonctionnement attendu par le client



(1) Au début de la description, la commande Mesure est supposée inactive. (2) Le séquenceur est dans un état d'attente au cours duquel aucune évolution du compteur réversible et du registre n'a lieu. (3) Dès que Mesure devient actif, chaque front d'horloge doit provoquer une décrémentation du compteur réversible. (4) Cette décrémentation se poursuit aussi longtemps que Mesure demeure actif ou que l'état 0 n'est pas atteint. (5) Si l'état 0 est atteint, les fronts d'horloge suivants doivent incrémenter le compteur pendant que Mesure reste actif. (6) Le retour à l'inactivité de Mesure doit conduire au chargement dans le registre de l'état de SensCroiss et de l'état du compteur ainsi qu'à une initialisation de ce dernier. (7) Le séquenceur se place à nouveau dans l'état d'attente.

2) Codage effectué



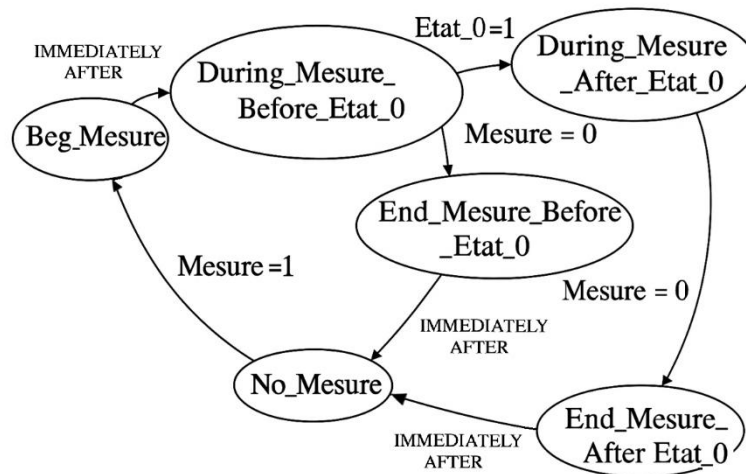
La principale différence est le remplacement de l'horloge FreqRef par l'horloge Clk (pas divisée) ce qui ici revient au même. Le principe même d'un séquenceur appelle une machine à états (noms des états ci-dessus) :

```

42 architecture Behavioral of Sequencer is
43
44     type STATE is (No_Mesure, Beg_Mesure, During_Mesure_Before_Etat_0, During_Mesure_After_Etat_0, End_Mesure_Before_Etat_0, End_Mesure_After_Etat_0);
45     signal CURRENT_STATE, NEXT_STATE : STATE:=No_Mesure;
46 begin

```


Graphe à états :



```

53 ---Combi pour det Etat_Futur
54 ETF : process (CURRENT_STATE, Mesure, Etat_0)
55 begin
56   -- valeur par défaut : dans le premier when si il n'y avait pas de else CURRENT_STATE ne serait pas affecté = erreur
57   -- l'autre solution est justement de le mettre
58   NEXT_STATE <= CURRENT_STATE;
59
60   case CURRENT_STATE is
61
62   when No_Mesure => if (Mesure='1') then NEXT_STATE <= Beg_Mesure; else NEXT_STATE <= No_Mesure; end if;
63
64   when Beg_Mesure => NEXT_STATE <= During_Mesure_Before_Etat_0;
65   ---Init_compt<= '1';
66
67   when During_Mesure_Before_Etat_0 =>
68     if (Mesure='0') then NEXT_STATE <= End_Mesure_Before_Etat_0;
69     elsif (Etat_0='1') then NEXT_STATE <= During_Mesure_After_Etat_0; end if; --Permet de créer un mini délai
70
71   when During_Mesure_After_Etat_0 =>
72     if (Mesure='0') then NEXT_STATE <= End_Mesure_After_Etat_0; end if;
73
74   when End_Mesure_Before_Etat_0 => NEXT_STATE <= No_Mesure;
75   when End_Mesure_After_Etat_0 => NEXT_STATE <= No_Mesure;
76   when others => NEXT_STATE <= No_Mesure; --A METTRE !
77
78   end case;
79 end process ETF;
  
```

Ici les sorties de notre machine à état sont Charg_Reg, Init_Compt, Valid_Compt et SensCroiss. Etant donné que ces sorties dans mon code ne dépendent que de l'état de notre machine à état, la machine fabriquée est de Moore.

Beg_Mesure laisse un coup d'horloge pour faire réinitialiser le compteur réversible tandis que les 2 End_Mesure font pareil pour Charg_Reg.

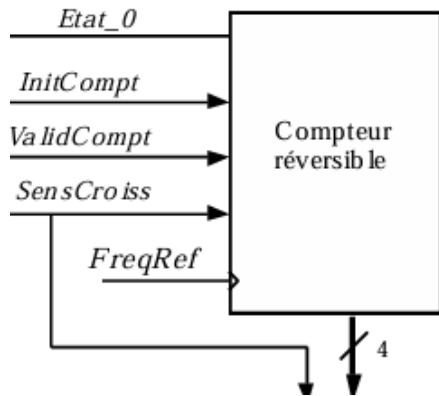
Afer_Etat_0 permet de spécifier que SensCroiss = '1'. (Permet de ne pas à garder Etat_0='1' pendant SensCroiss = '1' => Pas besoin de le réinitialiser + Permet d'avoir une machine de Moore non de Mealy [apparemment souhaitable car plus robuste]).

```

---Combi pour sortie fait entre chaque états
---Init_comp sert à reset le compteur réversible
Init_comp<= '1' when (CURRENT_STATE=Beg_Mesure) else '0';
SensCroiss <= '1' when (CURRENT_STATE=During_Mesure_After_Etat_0 or CURRENT_STATE=End_Mesure_After_Etat_0) else '0'; -- décalage de 1 attendu
Charg_Reg<= '1' when (CURRENT_STATE=End_Mesure_Before_Etat_0 or CURRENT_STATE=End_Mesure_After_Etat_0) else '0'; -- Nécess car Etat_0 se signale par qu'un front montant
Valid_Compt<= '1' when ((CURRENT_STATE=Beg_Mesure) or (CURRENT_STATE=During_Mesure_Before_Etat_0) or (CURRENT_STATE=During_Mesure_After_Etat_0)) else '0';
end Behavioral;
  
```

C-Compteur réversible

1) Fonctionnement attendu par le client



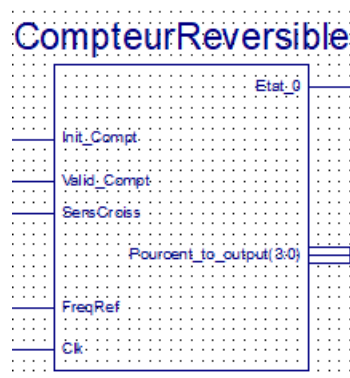
Il s'agit d'un compteur synchrone, réversible, de code binaire naturel, déclenché sur front montant d'horloge.

```

Si1 InitCompt est actif
  Le compteur est positionné à l'état d'équivalent décimal 100
Sinon1
  Si2 ValidCompt est actif
    Si3 SensCroiss est actif
      Le compteur est incrémenté à chaque période d'horloge
    Sinon3
      Le compteur est décrémenté
    FinSi3
  Sinon2
    L'état du compteur est maintenu inchangé
  FinSi2
Finsi1
  
```

Etat_0 est actif au cours de l'état 0 du compteur, quels que soient InitCompt, ValidCompt et SensCroiss.

2) Codage effectué



Clk s'occupe de la synchronisation tandis que **FreqRef** est un signal activé pendant un coup d'horloge à une fréquence de 196Hz (à postéri je me rends compte que FreqRef est mal nommé, ça devrait plutôt être FreqRef_enable).

Pourcentage_to_output convertit le compteur en pourcentage ou "1111" (=> overflow) si le compteur est au-delà de 8 (soit au de là de +/- 8 %).

Mis à part cela le compteur suit le fonctionnement décrit par le client.

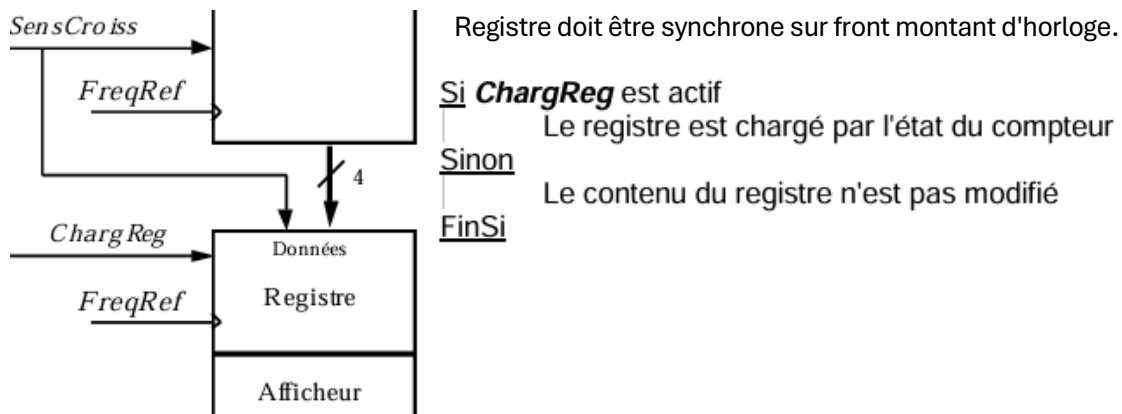
```

52 process (Clk, FreqRef, Init_Compt)
53 begin
54     ---Incré/Décré
55     ---rising_edge ne peut pas se mélanger avec autres choses,
56     ---sa propagation se fait dans des pistes en Or (meilleur conductivité) allouées séparé du circuit
57
58     if(rising_edge(Clk)) then
59         if(Init_Compt='1') then
60             compteur<=100;
61             Etat_0<='0';
62         else
63             if(compteur=0) then Etat_0 <= '1'; end if; ---Prise en compte le retard de la machine à l'arrêt
64             if(Valid_compt='1') then
65                 if(SensCroiss='0' and FreqRef='1') then compteur <= compteur - 1; --- SensCroiss
66                 elsif(FreqRef='1') then compteur<=compteur+1;
67             end if;
68         end if;
69     end if;
70 end if;
71 end process;
72
73 --- PARTIE Combinatoire
74 --- Etat_0 Management (Multiplexeur)
75 Pourcent_to_output <= std_logic_vector(to_unsigned(compteur,4)) when (compteur <= 8) else "1111";
76
77 end Behavioral;

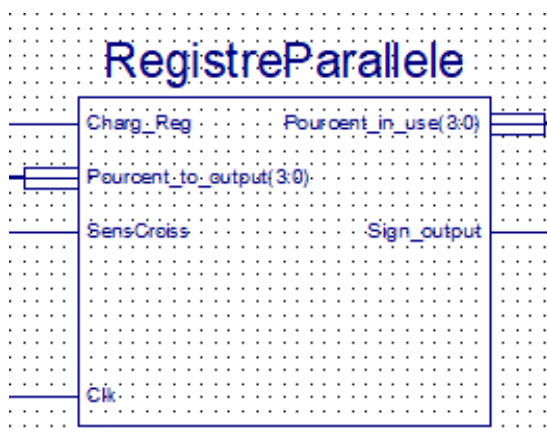
```

D-Registre parallèle

1) Fonctionnement attendu par le client

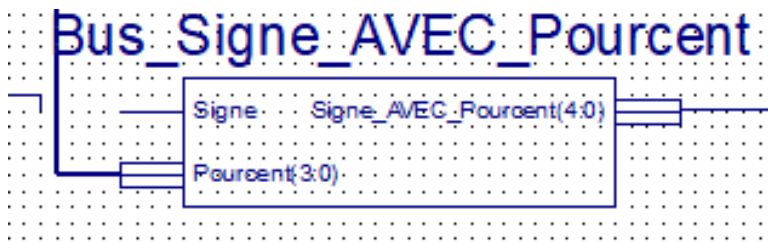


2) Codage effectué



Le fonctionnement est tel qu'attendu. Etant donné que l'on doit transmettre le pourcentage et le signe à l'afficheur on les voit à l'entrée et à la sortie du bloc registre parallèle.

E-Bus_Signe_AVEC_Pourcent



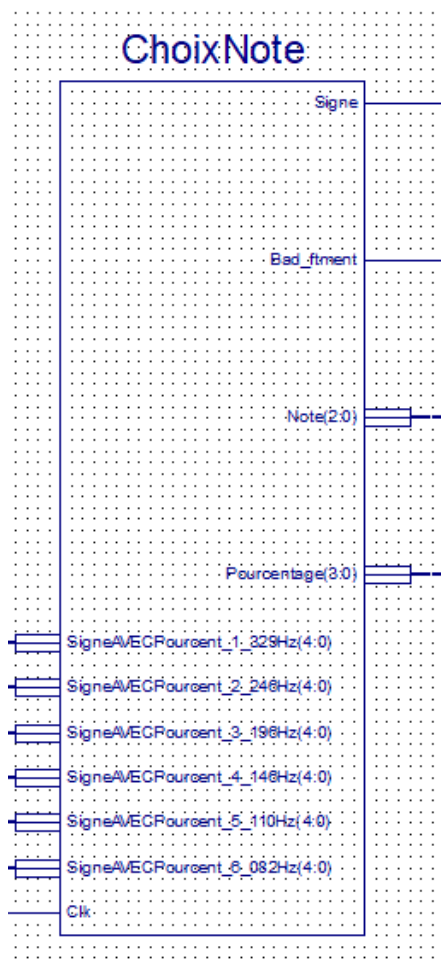
```

36 architecture Behavioral of Bus_Signe_AVEC_Pourcent is
37
38 begin
39
40     Signe_AVEC_Pourcent <= Signe & Pourcent;
41
42 end Behavioral;

```

Ce bloc a été ajouté uniquement pour des raisons esthétiques. Sinon au niveau des entrées de choixNote serait illisibles (les entrées/sorties sur le schéma bloc créé par Xilinx sont triés – bus en bas/ port logiques en haut).

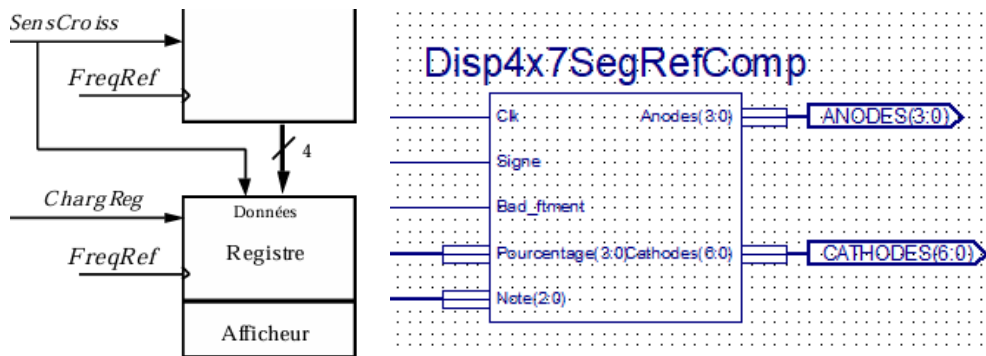
F-ChoixNote



ChoixNote est un bloc permettant de paralléliser les différents résultats pour différentes notes (Voir III – Conclusion). On sait que les notes sont exclusives, si une est valide les autres ne le seront pas. Ainsi, il suffit juste de savoir laquelle n'est pas "X1111", en déduire la note et sélectionner le pourcentage et le signe de la note valide. (Voir code en annexe)

Si aucune note est valide Bad_ftment est activé (techniquement mettre Pourcentage à 1111 suffirait mais je trouve que c'est plus clair pour moi).

G-Disp4x7SegRefComp



Comme ChoixNote, ce bloc n'est pas précisé par le client. Elle est basée sur un fichier .vhd du même nom trouvable dans notre section de projet Moodle. Elle permet de convertir les datas en entrée en chiffres affichés sur le compteur.

```

46
47 architecture Behavioral of Disp4x7SegRefComp is
48
49 signal DataA : STD_LOGIC_VECTOR (3 downto 0);
50 signal DataB : STD_LOGIC_VECTOR (3 downto 0);
51 signal DataC : STD_LOGIC_VECTOR (3 downto 0);
52 signal DataD : STD_LOGIC_VECTOR (3 downto 0);
53
54 type MATRICE is array (0 to 15) of STD_LOGIC_VECTOR (6 downto 0);
55
56 signal Data4x4Bits : STD_LOGIC_VECTOR (15 downto 0) := x"0000";
57 signal Adressage : STD_LOGIC_VECTOR (3 downto 0) := x"0";
58
59 -- signal Comptage : STD_LOGIC_VECTOR (3 downto 0) := x"0";
60 -- alias Decodage is Comptage (3 downto 2);
61 signal Comptage : STD_LOGIC_VECTOR (17 downto 0) := (others => '0');
62 alias Decodage is Comptage (17 downto 16);
63
64 constant CODAGE7SEGMENTS : MATRICE := ("1000000", -- Caractere "0"
65                                         "1111001", -- Caractere "1"
66                                         "0100100", -- Caractere "2"
67                                         "0110000", -- Caractere "3"
68                                         "0011001", -- Caractere "4"
69                                         "0010010", -- Caractere "5"
70                                         "0000010", -- Caractere "6"
71                                         "1111000", -- Caractere "7"
72                                         "0000000", -- Caractere "8"
73                                         "0010000", -- Caractere "9"
74                                         "0001000", -- Caractere "A"
75                                         "0000011", -- Caractere "b"
76                                         "1000110", -- Caractere "C"
77                                         "0101011", -- Caractere "n"
78                                         "0111111", -- Caractere "-"
79                                         "1111111" -- Caractere " "
80                                         );

```

Lorsque aucun signal est mesuré l'affichage doit renvoyer "n0" (pour « no signal ») avec un éventuel signe qui pourrait potentiellement aider à se repérer (si on n'a pas les fréquences par cœur en tête).

Sinon le premier digit est pour la note, le second est vide, le troisième est le signe et le quatrième est le pourcentage.

Exemple :

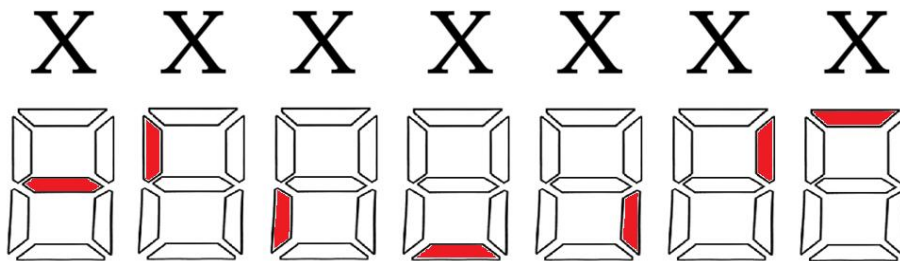


On détecte la note 1, de signe + (représenté, troisième position, par un caractère vide) et de pourcentage relatif 5%.

DataX permet de choisir un caractère à afficher dans le tableau CODAGE7SEGMENTS.

Exemple : "1111" -> Caractère vide ; "1001" -> C (inutilisé) ; "0011" -> 4

Chaque entrée du tableau est associée à un mot de 7 bits. Ces 7 bits contrôlent les 7 cathodes internes à l'afficheur d'une du FPGA. Lorsqu'un bit est mis à 1, cela signifie que la cathode associée est mise à 0. L'ordre des segments est décrit ci-dessous (ici un exemple est donné pour le caractère 1) :



Les 4 digits partagent les mêmes segments donc on doit afficher qu'un digit à la fois. C'est pour cela que l'on affiche le contenu d'un data sur une seule anode d'où la présence de décodage et adressage.

```

91 Anodes <= "1110" when Decodage = "00" else
92     "1101" when Decodage = "01" else
93     "1011" when Decodage = "10" else
94     "0111";
95
96 DataA <= Pourcentage; --Pourcentage (void se fait automatiquement + haut)
97 DataB <= "1111" when (Signe='0') else "1110"; -- rien ou -
98 DataC <= "1111" when (Bad_ftment='0') else "0000"; -- rien ou le o de no
99 DataD <= "0"&Note when (Bad_ftment='0') else "1101"; -- la note ou le n de no
100
101 Data4x4Bits <= DataD & DataC & DataB & DataA;
102 Adressage <= Data4x4Bits (((CONV_INTEGER(Decodage) * 4) + 3) downto (CONV_INTEGER(Decodage) * 4));
103 Cathodes <= CODAGE7SEGMENTS (CONV_INTEGER(Adressage));
104
105 end Behavioral;

```

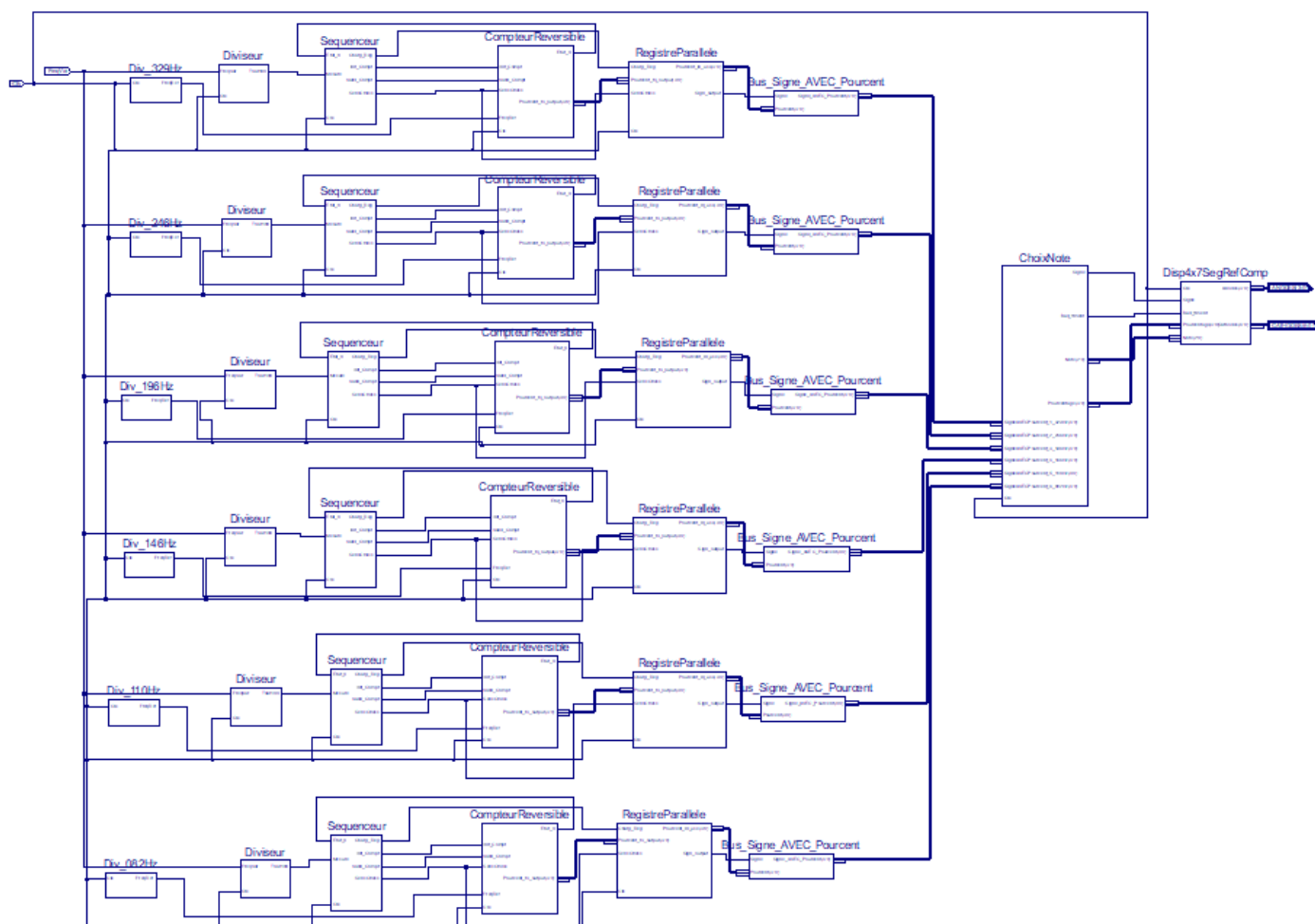
Comme pour les cathodes, 0 indique anode allumée. L'adressage se déplace de blocs de 4 bits en 4 bits pour un certain nombre de coups d'horloge ; ainsi une anode ($\leftrightarrow$ un digit) est associée à un DataX.

III – Conclusion

Finalement :

- Notre compilation a comme seul warning le fait que j'aurais pu mettre un unique bloc diviseur dans le schéma (à juste titre).
- La demande du client a été répondu mais FreqRef doit être renommé en FreqRef_enable plutôt. Mieux encore on aurait dû essayer d'implémenter des pll's au lieu de diviser sur piste en cuivre pour mieux coller au design du client.

Schéma final du système :



Remarque : Les PINs sélectionnés sont tous de tension 3.3V.

PARTIE Analogique

La partie analogique du système est organisée en 4 blocs :

- Alimentation et régulateurs
- Micro à électret
- Filtrage
- Comparateur

1) Alimentation et régulateurs :

Le circuit est alimenté par une source externe de 12 V, utilisée directement pour l'alimentation des amplificateurs TL081 et TL084.

Afin d'adapter les niveaux de tension aux autres composants du montage, deux régulateurs ont été intégrés au circuit : le L78L09 et le L78L33.

Le régulateur L78L09 permet d'abaisser la tension de 12 V à 9 V, tension nécessaire à l'alimentation du microphone à électret.

À partir de cette tension de 9 V, le régulateur L78L33 génère une tension 3,3 V, utilisée comme une tension (pull-up) sur la sortie du comparateur LM311.

Pour chaque régulateur, deux condensateurs ont été placés une en entrée et l'autre en sortie afin d'assurer le découplage.



Les deux régulateurs sont déjà commandés

2) Micro à électret :

Le signal sonore de la guitare est capté à l'aide d'un microphone à électret, qui transforme le son en un signal électrique de faible amplitude. Ce signal constitue l'entrée de la chaîne de traitement analogique.

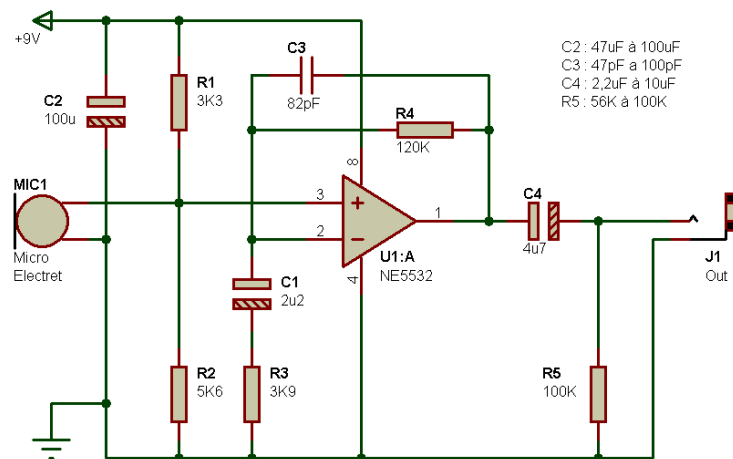
Le microphone à électret nécessite une polarisation en tension continue pour fonctionner correctement. Celle-ci est réalisée à l'aide d'un réseau de résistances alimenté en 9 V, qui

assure à la fois l'alimentation du microphone et la création d'un point de repos. On travaille en mono-tension $[0, 9V]$ donc entrée $(4,5 + V_m(t))$ ou $4,5 - V_m(t)$ où V_m est l'amplitude.

Le signal issu du microphone est amplifié à l'aide d'un amplificateur TL081 monté en configuration non-inverseuse. Le gain est déterminé par le réseau de résistances de contre-réaction et permet d'obtenir une amplitude suffisante.

Des condensateurs sont intégrés au montage afin d'assurer la stabilité de l'amplificateur et de limiter les perturbations haute fréquence. Un condensateur de liaison en sortie permet également de supprimer la composante continue du signal.

Ce bloc délivre ainsi un signal audio amplifié, prêt à être filtré puis comparé dans les parties suivantes.



Micro à électret

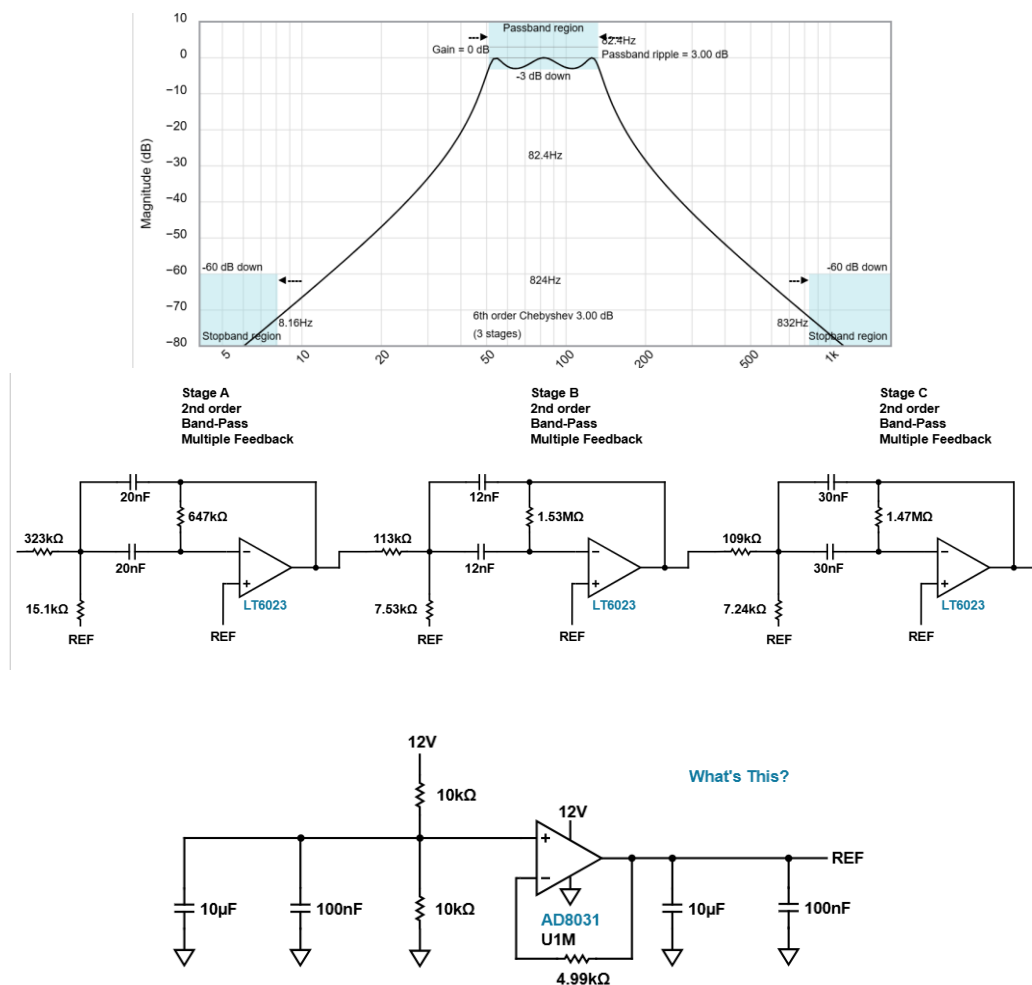
3) Filtrage :

Pour le moment, nous travaillons uniquement avec la note Mi (82.4 Hz).

Le signal issu du préamplificateur contient la fréquence de la note jouée ainsi que des harmoniques. Un filtre passe-bande est donc utilisé afin de ne conserver que la bande de fréquence utile correspondant à la note à accorder.

Le filtrage (d'ordre 6) est réalisé à l'aide de plusieurs étages actifs du second ordre montés en cascade et alimentés en mono-tension [0,12V]. Une tension de référence V_{ref} est générée afin de servir de point milieu au signal, permettant aux amplificateurs de traiter un signal alternatif sans alimentation symétrique.

À la sortie de ce bloc, le signal filtré est centré autour de $V_{ref} = 6V$ et principalement composé de la fréquence fondamentale de la note sélectionnée. Il est alors prêt à être appliqué au comparateur pour la mise en forme logique.



4) Comparateur :

Le comparateur transforme le signal analogique sinusoïdal en un signal logique carré (0 – 3.3V), qui va rentrer dans la carte FPGA.

Il compare le signal audio amplifié et filtré avec la tension de référence ($V_{ref} = 6V$).

Si $V+ > V_{ref}$: sortie 3.3V.

Sinon, sortie 0V.

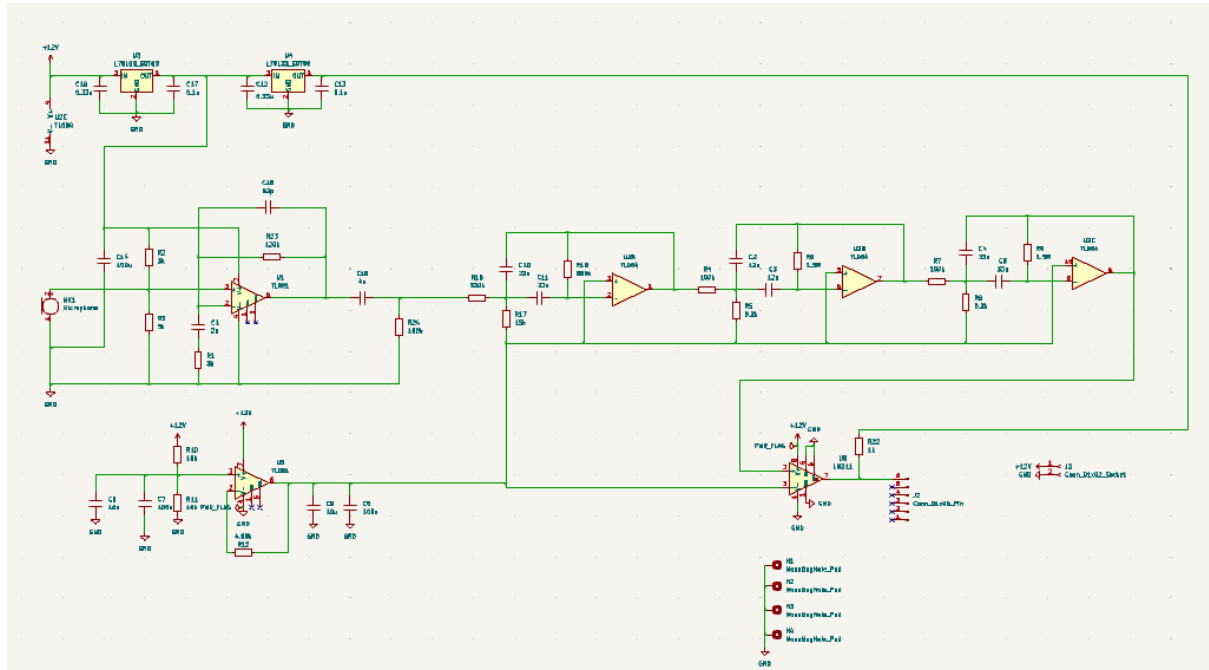
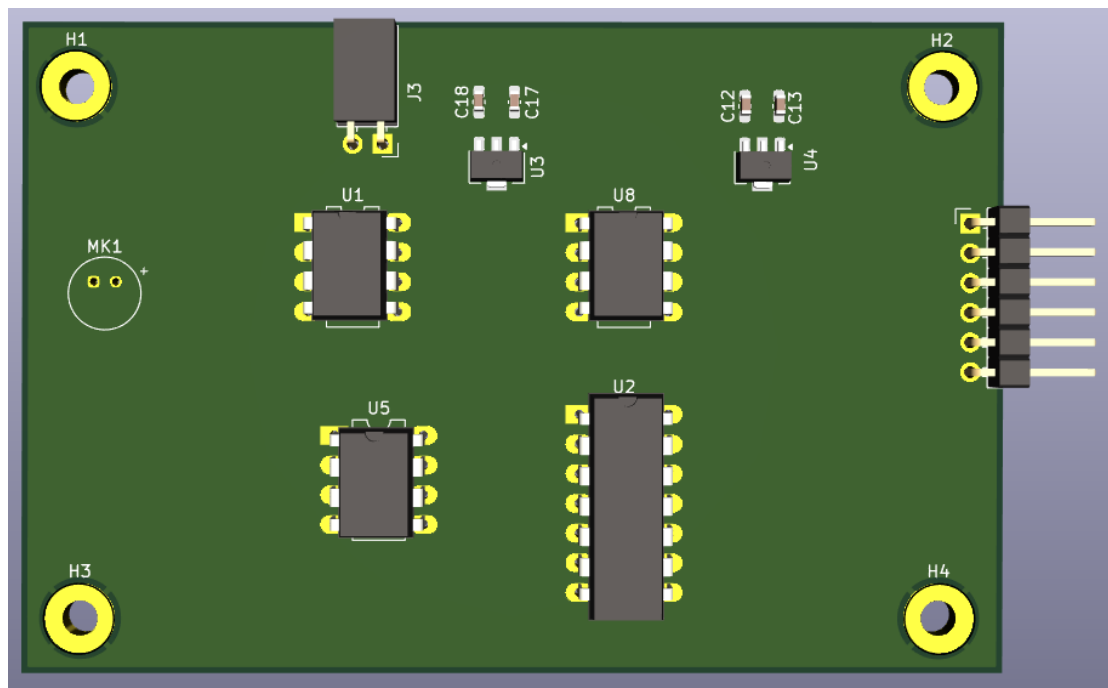


Schéma complet (schématique)



Vision 3D de la carte (nous n'avons pas encore terminé la connexion et le routage, résistances et condensateurs pas encore placés)

DOCUMENTATIONS

Moodle : Projet JOSSE

Pour décider de l'emplacement des PINs : [Basys2 Reference Manual | PDF](#)

ANNEXE

CODE DIVISEUR (PS : Le bloc Diviseur (non Hz en revanche) est dupliqué dans le schéma alors qu'un seul est suffisant. Heureusement la synthèse les optimise.)

```

20 library IEEE;
21 use ieee.numeric_std.all;
22 use IEEE.STD_LOGIC_1164.ALL;
23 use IEEE.STD_LOGIC_ARITH.ALL;
24 use IEEE.STD_LOGIC_UNSIGNED.ALL;
25
26 ---- Uncomment the following library declaration if instantiating
27 ---- any Xilinx primitives in this code.
28 --library UNISIM;
29 --use UNISIM.VComponents.all;
30
31 ---
32
33 entity Diviseur is
34     Port ( FreqVar : in  STD_LOGIC;
35           Clk : in STD_LOGIC;
36           TVar100 : out STD_LOGIC := '0'
37     );
38 end Diviseur;
39
40 ---Attention le retour a zero se fait par Overflow
41 architecture Behavioral of Diviseur is
42     signal compteur : natural :=0;
43     signal Q : std_logic:='0';
44     signal Qavant : std_logic:='0';
45
46 begin
47
48     ---Incrémentation
49     process(Clk)
50     begin
51         if(rising_edge(Clk)) then
52             if(compteur<=128) then
53                 if(Q /= Qavant) then
54                     compteur <= compteur+1;
55                     Qavant <= Q;
56                 else Q<=FreqVar; end if;
57             else compteur <=0; end if;
58         end if;
59     end process;
60
61     ---Créneau Tvar100
62     TVar100 <= '1' when (compteur < 100) else
63         '0';
64 end Behavioral;

```

CODE Div_196Hz

```

20 library IEEE;
21 use IEEE.STD_LOGIC_1164.ALL;
22 use IEEE.STD_LOGIC_ARITH.ALL;
23 use IEEE.STD_LOGIC_UNSIGNED.ALL;
24
25 ---- Uncomment the following library declaration if instantiating
26 ---- any Xilinx primitives in this code.
27 --library UNISIM;
28 --use UNISIM.VComponents.all;
29
30 entity Div_196Hz is
31     Port ( Clk : in  STD_LOGIC;
32           FreqRef : out STD_LOGIC:= '0'); ---FreqRef agit comme un signal enable
33 end Div_196Hz;
34
35 architecture Behavioral of Div_196Hz is
36     signal compteur_reel : natural := 0;
37     constant SEUIL_196Hz : natural :=63776; --127551.02/2 car le M6 compte 2 fois plus lentement que prévu
38
39 begin
40
41 process(Clk)
42 begin
43     if(rising_edge(Clk)) then
44         if(compteur_reel=SEUIL_196Hz) then compteur_reel<=0; FreqRef <= '1';
45         else compteur_reel<=compteur_reel+1; FreqRef <= '0'; end if;
46     end if;
47 end process;
48 end Behavioral;
49
50

```

CODE Séquenceur

```

20 library IEEE;
21 use IEEE.STD_LOGIC_1164.ALL;
22 use IEEE.STD_LOGIC_ARITH.ALL;
23 use IEEE.STD_LOGIC_UNSIGNED.ALL;
24
25 ---- Uncomment the following library declaration if instantiating
26 ---- any Xilinx primitives in this code.
27 --library UNISIM;
28 --use UNISIM.VComponents.all;
29
30 entity Sequenceur is
31     Port ( Etat_0 : in  STD_LOGIC;
32           Mesure : in  STD_LOGIC;
33           Clk : in  STD_LOGIC;
34           Init_Compt : out STD_LOGIC;
35           Charg_Reg : out STD_LOGIC;
36           Valid_Compt : out STD_LOGIC:= '0';
37           SensCroiss : out STD_LOGIC);
38 end Sequenceur;

```



```

42 architecture Behavioral of Sequenceur is
43
44     type STATE is (No_Mesure, Beg_Mesure, During_Mesure_Before_Etat_0, During_Mesure_After_Etat_0, End_Mesure_Before_Etat_0, End_Mesure_After_Etat_0);
45     signal CURRENT_STATE, NEXT_STATE : STATE:=No_Mesure;
46 begin
47     ---Synchro
48     CLK_state : process (Clk)
49     begin
50         if rising_edge(Clk) then CURRENT_STATE<=NEXT_STATE; end if;
51     end process CLK_state;
52
53     ---Combi pour det Etat_Futur
54     EIF : process (CURRENT_STATE, Mesure, Etat_0)
55     begin
56         -- valeur par défaut : dans le premier when si il n'y avait pas de else CURRENT_STATE ne serait pas affecté = erreur
57         -- l'autre solution est justement de le mettre
58         NEXT_STATE <= CURRENT_STATE;
59
60         case CURRENT_STATE is
61
62         when No_Mesure => if (Mesure='1') then NEXT_STATE <=Beg_Mesure; else NEXT_STATE <=No_Mesure; end if;
63
64         when Beg_Mesure => NEXT_State <=During_Mesure_Before_Etat_0;
65         ---Init_compt<= '1';
66
67         when During_Mesure_Before_Etat_0 =>
68             if (Mesure='0') then NEXT_STATE <=End_Mesure_Before_Etat_0;
69             elsif (Etat_0='1') then NEXT_STATE<=During_Mesure_After_Etat_0; end if; --Permet de créer un mini délai
70
71         when During_Mesure_After_Etat_0 =>
72             if (Mesure='0') then NEXT_STATE <=End_Mesure_After_Etat_0; end if;
73
74         when End_Mesure_Before_Etat_0 => NEXT_STATE <=No_Mesure;
75         when End_Mesure_After_Etat_0 => NEXT_STATE <=No_Mesure;
76         when others => NEXT_STATE <= No_Mesure; --A METTRE !
77
78         end case;
79     end process EIF;
80     ---Combi pour sortie fait entre chaque état
81     ---Init_compt sert à reset le compteur réversible
82     Init_compt<= '1' when ((CURRENT_STATE=Beg_Mesure) else '0';
83     SensCroiss <= '1' when ((CURRENT_STATE=During_Mesure_After_Etat_0 or CURRENT_STATE=End_Mesure_After_Etat_0) else '0'; -- décalage de 1 attendu
84     Charg_Reg<= '1' when ((CURRENT_STATE=End_Mesure_Before_Etat_0 or CURRENT_STATE=End_Mesure_After_Etat_0) else '0'; -- Necoss car Etat_0 se signale par qu'un front montant
85     Valid_Compt<='1' when (((CURRENT_STATE=Beg_Mesure) or ((CURRENT_STATE=During_Mesure_Before_Etat_0) or ((CURRENT_STATE=During_Mesure_After_Etat_0)) else '0';
86
87 end Behavioral;

```

CODE Compteur réversible

```

22 library IEEE; use IEEE.STD_LOGIC_1164.ALL;
23 use IEEE.STD_LOGIC_ARITH.ALL;
24 use IEEE.STD_LOGIC_UNSIGNED.ALL;
25 use IEEE.numeric_std.all;
26
27 ---- Uncomment the following library declaration if instantiating ---- any Xilinx primitives in this code. --library UNISIM; --use UNISIM.VComponents.all;
28
29 entity CompteurReversible is
30     Port (
31         Init_Compt : in STD_LOGIC;
32         Valid_Compt : in STD_LOGIC;
33         SensCroiss : in STD_LOGIC; --- Devra être raccordé à 2 LEDs : une pour +(verte) l'autre pour -(rouge)
34         FreqRef : in STD_LOGIC; ---FreqRef est un signal "Enable"
35         Clk : in STD_LOGIC;
36         Etat_0 : out STD_LOGIC;
37         Pourcent_to_output : out STD_LOGIC_VECTOR(3 downto 0)
38     );
39 end CompteurReversible;
40
41 --- La conversion de pourcentage_to_output de 8bits->4bits a posteriori en PERMANENCE!!!! doit se faire ICI
42
43 ---ATTENTION
44 ---Le seuil pris est pour calculer 329.6Hz (pour passer à un autre il suffit de changer *) Voir cahier pour seuils
45 ---ATTENTION

```

```

47 architecture Behavioral of CompteurReversible is
48     signal compteur : natural :=100;
49
50 begin
51     --- Compteur Management (y mettre SensCroiss)
52     process(Clk,FreqRef,Init_Compt)
53     begin
54         ---Incré/Décré
55         ---rising_edge ne peut pas se mélanger avec autres choses,
56         ---sa propagation se fait dans des pistes en Or (meilleur conductivité) allouées séparé du circuit!
57
58         if(rising_edge(Clk)) then
59             if(Init_Compt='1') then
60                 compteur<=100;
61                 Etat_0<='0';
62             else
63                 if(compteur=0) then Etat_0 <= '1'; end if; ---Prise en compte le retard de la machine à état ne fait rien (compteur n'y change pas - trop court)
64                 if(Valid_compt='1') then
65                     if(SensCroiss='0' and FreqRef='1') then compteur <= compteur - 1; --- SensCroiss='0' => décomptage
66                     elsif(FreqRef='1') then compteur<=compteur+1;
67                     end if;
68                 end if;
69             end if;
70         end if;
71     end process;
72
73     --- PARTIE Combinatoire
74     --- Etat_0 Management (Multiplexeur)
75     Pourcent_to_output <= std_logic_vector(to_unsigned(compteur,4)) when (compteur <= 8) else "1111";
76
77 end Behavioral;

```

CODE Bus_Signe_AVEC_Pourcent

```

20 library IEEE;
21 use IEEE.STD_LOGIC_1164.ALL;
22 use IEEE.STD_LOGIC_ARITH.ALL;
23 use IEEE.STD_LOGIC_UNSIGNED.ALL;
24
25 ---- Uncomment the following library declaration if instantiating
26 ---- any Xilinx primitives in this code.
27 --library UNISIM;
28 --use UNISIM.VComponents.all;
29
30 entity Bus_Signe_AVEC_Pourcent is
31   Port ( Signe : in  STD_LOGIC;
32         Pourcent : in  STD_LOGIC_VECTOR(3 downto 0);
33         Signe_AVEC_Pourcent : out  STD_LOGIC_VECTOR(4 downto 0));
34 end Bus_Signe_AVEC_Pourcent;
35
36 architecture Behavioral of Bus_Signe_AVEC_Pourcent is
37
38 begin
39
40     Signe_AVEC_Pourcent <= Signe & Pourcent;
41
42 end Behavioral;

```

CODE Choix Note

```

20 library IEEE;
21 use IEEE.STD_LOGIC_1164.ALL;
22 use IEEE.STD_LOGIC_ARITH.ALL;
23 use IEEE.STD_LOGIC_UNSIGNED.ALL;
24
25 ---- Uncomment the following library declaration if instantiating
26 ---- any Xilinx primitives in this code.
27 --library UNISIM;
28 --use UNISIM.VComponents.all;
29
30 entity ChoixNote is
31   Port ( SigneAVECPourcent_1_329Hz : in  STD_LOGIC_VECTOR(4 downto 0);
32         SigneAVECPourcent_2_246Hz : in  STD_LOGIC_VECTOR(4 downto 0);
33         SigneAVECPourcent_3_196Hz : in  STD_LOGIC_VECTOR(4 downto 0);
34         SigneAVECPourcent_4_146Hz : in  STD_LOGIC_VECTOR(4 downto 0);
35         SigneAVECPourcent_5_110Hz : in  STD_LOGIC_VECTOR(4 downto 0);
36         SigneAVECPourcent_6_082Hz : in  STD_LOGIC_VECTOR(4 downto 0);
37         Clk : in  STD_LOGIC;
38         Note : out  STD_LOGIC_VECTOR(2 downto 0);
39         Pourcentage : out  STD_LOGIC_VECTOR(3 downto 0);
40         Signe : out  STD_LOGIC;
41         Bad_ftment : out  STD_LOGIC);
42
43 end ChoixNote;

```

```

45 architecture Behavioral of ChoixNote is
46
47 begin
48   process(Clk,SigneAVECPourcent_1_329Hz, SigneAVECPourcent_2_246Hz, SigneAVECPourcent_3_196Hz,
49     SigneAVECPourcent_4_146Hz, SigneAVECPourcent_5_110Hz, SigneAVECPourcent_6_082Hz)
50   begin
51     if(rising_edge(Clk)) then
52       if SigneAVECPourcent_1_329Hz(3 downto 0) /= "1111" then
53         Pourcentage <= SigneAVECPourcent_1_329Hz(3 downto 0);
54         Note <= "001";
55         Signe <= SigneAVECPourcent_1_329Hz(4);
56         Bad_ftment <= '0';
57       elsif SigneAVECPourcent_2_246Hz(3 downto 0) /= "1111" then
58         Pourcentage <= SigneAVECPourcent_2_246Hz(3 downto 0);
59         Note <= "010";
60         Signe <= SigneAVECPourcent_2_246Hz(4);
61         Bad_ftment <= '0';
62       elsif SigneAVECPourcent_3_196Hz(3 downto 0) /= "1111" then
63         Pourcentage <= SigneAVECPourcent_3_196Hz(3 downto 0);
64         Note <= "011";
65         Signe <= SigneAVECPourcent_3_196Hz(4);
66         Bad_ftment <= '0';
67       elsif SigneAVECPourcent_4_146Hz(3 downto 0) /= "1111" then
68         Pourcentage <= SigneAVECPourcent_4_146Hz(3 downto 0);
69         Note <= "100";
70         Signe <= SigneAVECPourcent_4_146Hz(4);
71         Bad_ftment <= '0';
72       elsif SigneAVECPourcent_5_110Hz(3 downto 0) /= "1111" then
73         Pourcentage <= SigneAVECPourcent_5_110Hz(3 downto 0);
74         Note <= "101";
75         Signe <= SigneAVECPourcent_5_110Hz(4);
76         Bad_ftment <= '0';
77       elsif SigneAVECPourcent_6_082Hz(3 downto 0) /= "1111" then
78         Pourcentage <= SigneAVECPourcent_6_082Hz(3 downto 0);
79         Note <= "110";
80         Signe <= SigneAVECPourcent_6_082Hz(4);
81         Bad_ftment <= '0';
82       else
83         Pourcentage <= "1111"; -- pour signaler aucun signal détecté
84         Bad_ftment <= '1';
85
86       end if;
87     end if;
88   end process;
89 end Behavioral;

```

CODE Disp4x7SegRefComp

Avec commentaires

```

33 entity Disp4x7SegRefComp is
34   Port (
35     Clk : in std_logic;
36     --DataA : in STD_LOGIC_VECTOR (3 downto 0);
37     --DataB : in STD_LOGIC_VECTOR (3 downto 0);
38     --DataC : in STD_LOGIC_VECTOR (3 downto 0);
39     --DataD : in STD_LOGIC_VECTOR (3 downto 0);
40     Pourcentage : in STD_LOGIC_VECTOR (3 downto 0) := "0001"; ---Pourcentage
41     Signe : in STD_LOGIC := '1'; --Signe
42     Bad_ftment : in STD_LOGIC;
43     Note : in STD_LOGIC_VECTOR (2 downto 0);
44     Anodes : out STD_LOGIC_VECTOR (3 downto 0);
45     Cathodes : out STD_LOGIC_VECTOR (6 downto 0));
46 end Disp4x7SegRefComp;
47
48 architecture Behavioral of Disp4x7SegRefComp is
49   signal DataA : STD_LOGIC_VECTOR (3 downto 0);
50   signal DataB : STD_LOGIC_VECTOR (3 downto 0);
51   signal DataC : STD_LOGIC_VECTOR (3 downto 0);
52   signal DataD : STD_LOGIC_VECTOR (3 downto 0);
53
54   type MATRICE is array (0 to 15) of STD_LOGIC_VECTOR (6 downto 0); --tochange
55
56   signal Data4x4Bits : STD_LOGIC_VECTOR (15 downto 0) := x"0000";
57   signal Adressage : STD_LOGIC_VECTOR (3 downto 0) := x"0";
58
59   -- signal Comptage : STD_LOGIC_VECTOR (3 downto 0) := x"0";
60   -- alias Decodage is Comptage (3 downto 2);
61   signal Comptage : STD_LOGIC_VECTOR (17 downto 0) := (others => '0'); --Coupé de 19->17 car 2 dernières broches servent à rien
62   alias Decodage is Comptage (17 downto 16);
63   --(code de Gray: On a toujours une répartition équilibrée de "00"/"01"/"10" et "11")
64
65   constant CODAGE7SEGMENTS : MATRICE := (
66     "1000000", -- Caractere "0"
67     "1111001", -- Caractere "1"
68     "0100100", -- Caractere "2"
69     "0110000", -- Caractere "3"
70     "0011001", -- Caractere "4"
71     "0010010", -- Caractere "5"
72     "0000010", -- Caractere "6"
73     "1111000", -- Caractere "7"
74     "0000000", -- Caractere "8"
75     "0010000", -- Caractere "9"
76     "0001000", -- Caractere "A"
77     "0000011", -- Caractere "b"
78     "1000110", -- Caractere "C"
79     "0101011", -- Caractere "n"
80     "0111111", -- Caractere "-"
81     "1111111" -- Caractere " "
82   );

```

Sans commentaires

```

33 entity Disp4x7SegRefComp is
34     Port (      Clk : in std_logic;
35              --DataA : in  STD_LOGIC_VECTOR (3 downto 0);
36              --DataB : in  STD_LOGIC_VECTOR (3 downto 0);
37              --DataC : in  STD_LOGIC_VECTOR (3 downto 0);
38              --DataD : in  STD_LOGIC_VECTOR (3 downto 0);
39              Pourcentage : in  STD_LOGIC_VECTOR (3 downto 0) := "0001"; ---Pourcentage
40              Signe : in  STD_LOGIC := '1'; --Signe
41              Bad_ftment : in  STD_LOGIC;
42              Note : in  STD_LOGIC_VECTOR (2 downto 0);
43              Anodes : out  STD_LOGIC_VECTOR (3 downto 0);
44              Cathodes : out  STD_LOGIC_VECTOR (6 downto 0));
45 end Disp4x7SegRefComp;
46
47 architecture Behavioral of Disp4x7SegRefComp is
48
49     signal DataA : STD_LOGIC_VECTOR (3 downto 0);
50     signal DataB : STD_LOGIC_VECTOR (3 downto 0);
51     signal DataC : STD_LOGIC_VECTOR (3 downto 0);
52     signal DataD : STD_LOGIC_VECTOR (3 downto 0);
53
54     type MATRICE is array (0 to 15) of STD_LOGIC_VECTOR (6 downto 0);|
55
56     signal Data4x4Bits : STD_LOGIC_VECTOR (15 downto 0) := x"0000";
57     signal Adressage : STD_LOGIC_VECTOR (3 downto 0) := x"0";
58
59     -- signal Comptage : STD_LOGIC_VECTOR (3 downto 0) := x"0";
60     -- alias Decodage is Comptage (3 downto 2);
61     signal Comptage : STD_LOGIC_VECTOR (17 downto 0) := (others => '0');
62     alias Decodage is Comptage (17 downto 16);
63
64     constant CODAGE7SEGMENTS : MATRICE := ("1000000", -- Caractere "0"
65                                             "1111001", -- Caractere "1"
66                                             "0100100", -- Caractere "2"
67                                             "0110000", -- Caractere "3"
68                                             "0011001", -- Caractere "4"
69                                             "0010010", -- Caractere "5"
70                                             "0000010", -- Caractere "6"
71                                             "1111000", -- Caractere "7"
72                                             "0000000", -- Caractere "8"
73                                             "0010000", -- Caractere "9"
74                                             "0001000", -- Caractere "A"
75                                             "0000011", -- Caractere "b"
76                                             "1000110", -- Caractere "C"
77                                             "0101011", -- Caractere "n"
78                                             "0111111", -- Caractere "-."
79                                             "1111111"  -- Caractere " "
80                                             );

```

```

63
64 constant CODAGE7SEGMENTS : MATRICE := ("1000000", -- Caractere "0"
65     "1111001", -- Caractere "1"
66     "0100100", -- Caractere "2"
67     "0110000", -- Caractere "3"
68     "0011001", -- Caractere "4"
69     "0010010", -- Caractere "5"
70     "0000010", -- Caractere "6"
71     "1111000", -- Caractere "7"
72     "0000000", -- Caractere "8"
73     "0010000", -- Caractere "9"
74     "0001000", -- Caractere "A"
75     "0000011", -- Caractere "b"
76     "1000110", -- Caractere "C"
77     "0101011", -- Caractere "n"
78     "0111111", -- Caractere "-"
79     "1111111" -- Caractere " "
80     );
81 begin
82
83 process (CLK)
84     begin
85         if (CLK' event and CLK = '1') then
86             Comptage <= Comptage + '1';
87         end if;
88     end process;
89
90
91 Anodes <= "1110" when Decodage = "00" else
92     "1101" when Decodage = "01" else
93     "1011" when Decodage = "10" else
94     "0111";
95
96 DataA <= Pourcentage; --Pourcentage (void se fait automatiquement + haut)
97 DataB <= "1111" when (Signe='0') else "1110"; -- rien ou -
98 DataC <= "1111" when (Bad_ftment='0') else "0000"; -- rien ou le o de no
99 DataD <= "0"&Note when (Bad_ftment='0') else "1101"; -- la note ou le n de no
100
101 Data4x4Bits <= DataD & DataC & DataB & DataA;
102 Adressage <= Data4x4Bits (((CONV_INTEGER(Decodage) * 4) + 3) downto (CONV_INTEGER(Decodage) * 4));
103 Cathodes <= CODAGE7SEGMENTS (CONV_INTEGER(Adressage));
104
105 end Behavioral;

```